■ ALASSIO DATA CATALOG

Deployment Guide · Databricks App

Running as a Databricks App with Lakebase · Version 1.79.6

This guide describes how to run the Alassio Data Catalog as a Databricks App. It is the same code as on your own Ubuntu server – the Databricks edition is configuration only: the `app.yaml` file, Lakebase (PostgreSQL) as storage, sign-in through the platform, backups in a Unity Catalog volume and a Databricks job for scheduled tasks.

Allow about 60 minutes, part of it for permissions in the workspace.

Platform	Databricks Apps (the configuration targets Azure Databricks; AWS and GCP work the same way)
Runtime	Databricks Apps environment with Node.js 22.16 (the app needs at least 22.12), default size 2 vCPU / 6 GB
Storage	Lakebase (Autoscaling project or an existing Provisioned instance) · Unity Catalog volume for backups
Result	https://<app-url>/ with Databricks sign-in, MCP endpoint /mcp for AI agents, hourly background tasks

Caution: The Databricks edition is covered by automated tests (PostgreSQL, proxy sign-in, running without the inspector, task token, both Lakebase credential flows) but has not yet been run in a real workspace. Points to check on the first deployment are marked **[verify]** in the text.

Overview

Building block	On an Ubuntu server	As a Databricks App
Web server / TLS	nginx, Let's Encrypt	platform (app URL with HTTPS)
Sign-in	password + 2FA, optionally Entra ID	Databricks sign-in; the app reads the forwarded identity (`AUTH_MODE=header`)
Roles	in user management	first admins via `AUTH_BOOTSTRAP_ADMINS`, otherwise group → role mapping from Databricks groups
Database	MariaDB or PostgreSQL	Lakebase (PostgreSQL), password = OAuth token of the app's service principal
Source systems	Python inspector + native connectors	native connectors only; databases via Lakehouse Federation and the Unity Catalog import
Backups	local directory	Unity Catalog volume (`BACKUP_STORAGE=databricks`)
Scheduled tasks	built-in scheduler	a Databricks job calls the task endpoints hourly (`SCHEDULER=external`)
AI agents (MCP)	agent keys, OAuth, MCP_TOKEN	clients sign in to Databricks and act as the respective user

Step 1 · Prerequisites in the workspace

What	Why
Lakebase database: an Autoscaling project with a branch, a compute endpoint and a database (today's default) – or an existing Provisioned instance	catalog storage; the database attached as an app resource provides `PGHOST`, `PGPORT`, `PGUSER`, `PGDATABASE`, `PGSSLMODE`
Unity Catalog volume, e.g. `/Volumes/main/alassio/backups`	storage for the daily backups
Secret scope with the key `tasks-token` (and `smtp-pass` if mail is used)	token for the background tasks, SMTP password
Optional: Lakehouse Federation connections (SQL Server, Oracle, Snowflake, PostgreSQL …)	replaces the Python inspector: foreign databases appear as foreign catalogs and are loaded with the Unity Catalog import
Databricks CLI (current version) on your workstation, signed in to the workspace	creating secrets, uploading and deploying

```
openssl rand -base64 32
databricks secrets create-scope cortex
databricks secrets put-secret cortex tasks-token --string-value <token-from-openssl>
```

The scope name `cortex` is the default the job script expects (for another name use the job parameter `SECRET_SCOPE`).

Step 2 · Create the app and attach resources

In the workspace UI under **Compute › Apps**, create a new app of type “Custom”, e.g. named `alassio-catalog`, or use the CLI:

```
databricks apps create alassio-catalog
```

Then attach the following in the app settings under **Resources**:

Resource	Setting	Effect
Database (Lakebase)	the database from step 1, permission “Can connect and create”	creates a PostgreSQL role for the app's service principal; the app creates its tables on first start
Secret	scope `cortex`, key `tasks-token`, resource key `tasks-token`	read in `app.yaml` via `valueFrom: tasks-token` as `TASKS_TOKEN`
Volume	the backup volume, write access	target of `DATABRICKS_VOLUME_PATH`

Under **User authorization** enable the scope `iam.current-user:read`. The app uses it to read the Databricks groups of the signed-in person for the group → role mapping **[verify]**. Also note the app URL (e.g. `https://alassio-catalog-1234567890.1.azure.databricksapps.com`) – you need it in step 3.

Step 3 · Get the code and adapt app.yaml

```
git clone https://github.com/MevPR/Alassio-Data-Catalog.git
cd Alassio-Data-Catalog
cp deploy/databricks/app.yaml app.yaml
vim app.yaml
```

Without access to the repository, unzip `datahub-catalog-deploy.zip` instead and change into the `datahub-catalog` directory. Adapt in `app.yaml`:

Variable	Value
`LAKEBASE_ENDPOINT`	resource name of the compute endpoint: `projects/<id>/branches/<id>/endpoints/<id>` (Lakebase project › branch › Computes › "Copy resource name"). For a Provisioned instance use `LAKEBASE_INSTANCE` with the instance name instead
`AUTH_BOOTSTRAP_ADMINS`	e-mail address(es) of the first administrators, comma-separated – the first sign-in of these accounts creates them as admins
`AUTH_HEADER_DEFAULT_ROLES`	role of all other new users (default `guest`, read-only)
`APP_URL`	the app URL from step 2 (links in notifications)
`DATABRICKS_VOLUME_PATH`	path of the backup volume, e.g. `/Volumes/main/alassio/backups`
`DATABRICKS_ALLOWED_HOSTS`	host name of your own workspace (e.g. `adb-1234567890.1.azuredatabricks.net`) for the Unity Catalog import
`SMTP_*`	optional; without a reachable mail server notifications stay in the app's inbox **[verify]**

Fixed – do not change: `DB_DRIVER=postgres`, `DB_PASSWORD_PROVIDER=databricks`, `DB_SSL=1`, `AUTH_MODE=header`, `AUTH_GROUPS_FROM=databricks`, `COOKIE_SECURE=1`, `INSPECTOR_URL=""`, `DATA_DIR=/tmp/cortex`, `BACKUP_STORAGE=databricks`, `SCHEDULER=external`. The platform sets `DATABRICKS_HOST`, `DATABRICKS_CLIENT_ID`, `DATABRICKS_CLIENT_SECRET` and the port itself.

Step 4 · Upload and deploy

Option A – from a workspace folder:

```
databricks sync . /Workspace/Users/<your-e-mail>/alassio-catalog
databricks apps deploy alassio-catalog --source-code-path \
    /Workspace/Users/<your-e-mail>/alassio-catalog
```

`databricks sync` honours `.gitignore`; `node_modules`, `dist` and Python virtual environments are not uploaded. During deployment the platform runs `npm install` and `npm run build` and then starts the command from `app.yaml` (`node server.js`). The app automatically listens on the port Databricks assigns.

Option B – directly from Git: if the app is connected to the repository (private GitHub repository with a Git credential for the app's service principal), deploy a specific release by tag:

```
databricks apps deploy alassio-catalog --git-tag v1.79.6
```

> **Note:** With option B, `app.yaml` must be in the repository root. Because it contains workspace-specific values, use a dedicated branch for it or option A.

Step 5 · Check and first sign-in

- Open `https://<app-url>/healthz` in the browser: expected `"status":"ready"` and `"version":"1.79.6"`.
- Open the app's start page with an account from `AUTH_BOOTSTRAP_ADMINS` – there is no password screen, the platform signs you in; the account is created as admin.
- Under Admin › Users › “Group → role mapping”, map Databricks groups to roles (e.g. `data-stewards` → Data Steward). New users without a match get `AUTH_HEADER_DEFAULT_ROLES`.
- If start-up fails, check the app's logs (the app's **Logs** tab). “Lakebase credentials need …” means the database resource or `LAKEBASE_ENDPOINT` is missing.

Step 6 · Background tasks as a Databricks job

Without a long-running in-process scheduler, a job takes over the hourly tasks: Unity Catalog synchronisation, stewardship reminders, backup and purging old usage data.

Setting	Value
Task type	Python script, source workspace, path `…/alassio-catalog/deploy/databricks/run_tasks.py`
Compute	serverless
Parameters	`APP_URL` = app URL; optional `SECRET_SCOPE` (default `cortex`) and `TASKS` (default: `uc-sync,stewardship-reminders,backup,usage-purge`)
Schedule	hourly
Permissions	the identity the job runs as needs “Can use” on the app and read access to the secret scope

The script sends the token in the `X-Tasks-Token` header, because the `Authorization` header is taken by the platform sign-in. The backup writes the gzip'd catalog dump to the volume; the SQL dump is skipped because the app runtime has no `pg_dump`.

Step 7 · Connect Unity Catalog

- **Import** (Import › Unity Catalog): choose a sync profile or “App identity (Databricks Apps)” for a one-off import. The app's service principal needs `USE CATALOG`, `USE SCHEMA` and `SELECT` on the catalogs to import, and for governance tags also “Can use” on a SQL warehouse.
- **Scheduled synchronisation** (Admin › Backups & jobs › Scheduled synchronisation): a profile with catalog, target data hub, schedule and mode “Report only” or “Apply”; run by the job from step 6.
- **Write-back** (Export › Write back to UC): every user previews and writes back their own changes; admins additionally changes without a recorded author. The service principal needs `MODIFY` (or ownership) on the target objects and “Can use” on the profile's SQL warehouse; without a warehouse only comments and catalog properties are written.
- **Other sources**: OData/SAP SuccessFactors, Kafka Schema Registry, Power BI, Informatica, dbt, ODCS and CSV work as on the server; outbound connections follow the workspace's network policy **[verify]**.

Step 8 · AI agents (MCP)

Agents connect to `https://<app-url>/mcp` and sign in to Databricks via OAuth. The app acts as the forwarded person: admins and data stewards get write tools, everyone else read tools only. `MCP_TOKEN` and agent keys are not used in this edition.

Updates

Get the new version (`git pull` or unzip the new package), keep your own `app.yaml`, then upload and deploy again:

```
git pull
databricks sync . /Workspace/Users/<your-e-mail>/alassio-catalog
databricks apps deploy alassio-catalog --source-code-path \
    /Workspace/Users/<your-e-mail>/alassio-catalog
```

The app applies schema changes itself at start-up. Earlier deployments are listed with `databricks apps list-deployments alassio-catalog`.

Limits of this edition

- No password sign-in, no own two-factor authentication, no invitations and no own OAuth server – the platform handles this.
- No Python inspector: DuckDB files, data lake files and direct SQL/JDBC connections are not available; databases come through Lakehouse Federation.
- No persistent file system: `/tmp/cortex` holds upload temp files only.
- Mail only if the workspace allows outbound SMTP connections **[verify]**.

To verify on the first deployment

Point	How to tell	Fallback
Groups via `iam.current-user:read`	the group → role mapping applies on first sign-in	assign roles manually in Admin › Users
Lakebase token renewal	the app runs longer than an hour without database errors	check the logs; tokens are renewed 5 minutes before expiry
Outbound connections (APIs, SMTP)	source import or mail works	adjust the workspace network policy
Job access to the app	the job run shows HTTP 200 for all tasks	grant “Can use” on the app to the job identity